

Serverless Deployment for IMDb QLoRA Model

Overview

This guide covers deploying your fine-tuned IMDb QLoRA model in a serverless environment, eliminating the need for constant high-compute infrastructure while maintaining fast inference.

Deployment Options

1. Modal (Recommended for this use case)

Modal is purpose-built for ML inference with excellent cold start performance.

Pros:

- GPU/CPU auto-scaling
- No cold start for functions (warm pool by default)
- Built-in monitoring and logging
- Native support for large model weights
- Pay only for compute time used

Implementation:

```
# modal_deploy.py
import modal
from transformers import AutoTokenizer, AutoModelForCausalLM
from peft import PeftModel

app = modal.App("imdb-qlora-inference")

# Define container image with dependencies
imdb_image = modal.Image.debian_slim().pip_install(
    "torch>=2.0",
    "transformers>=4.30",
    "peft",
    "bitsandbytes",
)

@app.cls(
    image=imdb_image,
    gpu="A10G",
    container_idle_timeout=60,
)
class IMDBInference:
    def __enter__(self):
        self.device = "cuda"
        self.base_model = AutoModelForCausalLM.from_pretrained(
            "meta-llama/Llama-2-7b",
            device_map="auto",
            load_in_8bit=True,
        )
        self.model = PeftModel.from_pretrained(
```

```

        self.base_model,
        "./imdb-qlora-model" # Your fine-tuned adapter
    )
    self.tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-2-7b")

    @modal.method()
    def predict(self, text: str, max_tokens: int = 100) -> dict:
        inputs = self.tokenizer(text, return_tensors="pt").to(self.device)
        outputs = self.model.generate(
            *inputs,
            max_new_tokens=max_tokens,
            temperature=0.7,
            top_p=0.9,
        )
        result = self.tokenizer.decode(outputs[0], skip_special_tokens=True)
        return {"text": result}

    @app.function(image=imdb_image)
    def health_check():
        return {"status": "healthy"}

```

Deployment:

```

modal deploy modal_deploy.py
# Access via: modal.run(imdb_model.predict, text="...")

```

2. AWS Lambda + SageMaker

Best for AWS-native infrastructure.

Pros:

- Deep AWS integration
- Serverless pricing model
- Scales automatically
- Good for production workloads

Implementation:

```

# lambda_handler.py
import json
import boto3
import base64
from transformers import AutoTokenizer, AutoModelForCausalLM
from peft import PeftModel

sagemaker_client = boto3.client("sagemaker-runtime")

# Load model on cold start
model = None
tokenizer = None

```

```

def load_model():
    global model, tokenizer
    if model is None:
        model = AutoModelForCausalLM.from_pretrained(
            "meta-llama/Llama-2-7b",
            device_map="auto",
        )
        model = PeftModel.from_pretrained(model, "s3://bucket/imdb-qlora-adapter")
        tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-2-7b")
    return model, tokenizer

def lambda_handler(event, context):
    try:
        body = json.loads(event.get("body", "{}"))
        text = body.get("text", "")
        max_tokens = body.get("max_tokens", 100)

        model, tokenizer = load_model()

        inputs = tokenizer(text, return_tensors="pt")
        outputs = model.generate(*inputs, max_new_tokens=max_tokens)
        result = tokenizer.decode(outputs[0], skip_special_tokens=True)

        return {
            "statusCode": 200,
            "body": json.dumps({"prediction": result})
        }
    except Exception as e:
        return {
            "statusCode": 500,
            "body": json.dumps({"error": str(e)})
        }

```

3. Replicate (Easiest for quick deployment)

Replicate handles all infrastructure complexity.

Pros:

- Zero infrastructure management
- Built-in API versioning
- Automatic scaling
- No cold start issues

Implementation:

1. Create `cog.yaml` :

```

build:
  gpu: true
  python_version: "3.10"

```

```
python_packages:
  - torch>=2.0
  - transformers>=4.30
  - peft
  - bitsandbytes

predict: "predict.py:Predictor"
```

2. Create `predict.py` :

```
from cog import BasePredictor, Input, Path
from transformers import AutoTokenizer, AutoModelForCausalLM
from peft import PeftModel
import torch

class Predictor(BasePredictor):
    def setup(self):
        self.device = "cuda" if torch.cuda.is_available() else "cpu"
        self.base_model = AutoModelForCausalLM.from_pretrained(
            "meta-llama/Llama-2-7b",
            torch_dtype=torch.float16,
            device_map="auto",
        )
        self.model = PeftModel.from_pretrained(
            self.base_model,
            "imdb-qlora-adapter"
        )
        self.tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-2-7b")

    def predict(
        self,
        text: str = Input(description="Text to generate from"),
        max_tokens: int = Input(default=100, description="Max tokens"),
    ) -> str:
        inputs = self.tokenizer(text, return_tensors="pt").to(self.device)
        outputs = self.model.generate(
            *inputs,
            max_new_tokens=max_tokens,
            temperature=0.7,
        )
        return self.tokenizer.decode(outputs[0], skip_special_tokens=True)
```

3. Deploy:

```
cog push r8.im/username/imdb-qlora
```

4. Google Cloud Run

Good for multi-region deployments.

Implementation:

```
# main.py
from flask import Flask, request, jsonify
from transformers import AutoTokenizer, AutoModelForCausalLM
from peft import PeftModel
import torch

app = Flask(__name__)

# Load model on startup
device = "cuda" if torch.cuda.is_available() else "cpu"
base_model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-2-7b",
    device_map="auto",
)
model = PeftModel.from_pretrained(base_model, "gs://bucket/imdb-qlora")
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-2-7b")

@app.route("/predict", methods=["POST"])
def predict():
    data = request.json
    text = data.get("text", "")
    max_tokens = data.get("max_tokens", 100)

    inputs = tokenizer(text, return_tensors="pt").to(device)
    outputs = model.generate(*inputs, max_new_tokens=max_tokens)
    result = tokenizer.decode(outputs[0], skip_special_tokens=True)

    return jsonify({"prediction": result})

@app.route("/health", methods=["GET"])
def health():
    return jsonify({"status": "healthy"}), 200

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080)
```

Deploy with:

```
gcloud run deploy imdb-qlora --source . --platform managed --region us-central1 --
memory 16Gi --timeout 3600
```

Model Optimization for Serverless

1. Quantization

```
from transformers import AutoModelForCausalLM, BitsAndBytesConfig
```

```

quantization_config = BitsAndBytesConfig(
    load_in_8bit=True, # Reduces memory by 75%
    bnb_8bit_compute_dtype=torch.float16
)

model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-2-7b",
    quantization_config=quantization_config
)

```

2. Model Compression

```

# Using distillation or pruning
from transformers import distillation_utils

# Before deployment, distill to smaller model
smaller_model = distillation_utils.distill_model(
    teacher_model=model,
    student_model=AutoModelForCausalLM.from_pretrained("meta-llama/Llama-2-7b")
)

```

3. Cache Compiled Model

Store pre-loaded model in container image or cloud storage for faster cold starts.

Monitoring & Logging

1. CloudWatch Integration (AWS)

```

import logging
import watchtower

logger = logging.getLogger(__name__)
logger.addHandler(watchtower.CloudWatchLogHandler())

@app.route("/predict", methods=["POST"])
def predict():
    logger.info(f"Prediction request: {request.json}")
    try:
        result = model.generate(...)
        logger.info(f"Generated successfully: {len(result)} tokens")
        return jsonify({"result": result})
    except Exception as e:
        logger.error(f"Prediction failed: {str(e)}")
        return jsonify({"error": str(e)}), 500

```

2. Prometheus Metrics

```
from prometheus_client import Counter, Histogram, start_http_server
import time

prediction_counter = Counter("predictions_total", "Total predictions")
inference_time = Histogram("inference_seconds", "Inference time in seconds")

@app.route("/predict", methods=["POST"])
def predict():
    start_time = time.time()
    try:
        result = model.generate(...)
        prediction_counter.inc()
        inference_time.observe(time.time() - start_time)
        return jsonify({"result": result})
    except Exception as e:
        inference_time.observe(time.time() - start_time)
        raise
```

3. Structured Logging

```
import json
from datetime import datetime

def log_prediction(text, result, latency, tokens):
    log_entry = {
        "timestamp": datetime.utcnow().isoformat(),
        "input_length": len(text),
        "output_tokens": tokens,
        "latency_ms": latency * 1000,
        "model": "imdb-qlora"
    }
    print(json.dumps(log_entry))
```

Cost Comparison

Platform	Compute Cost	Cold Start	Best For
Modal	~\$0.50/GPU hour	<1s	General inference
AWS Lambda	~\$0.20 per 1M requests	10-30s	Batch jobs
Replicate	~\$0.001 per prediction	<1s	Public APIs
Google Cloud Run	~\$0.40/vCPU hour	5-10s	Web services
Together AI	~\$0.0002 per 1K tokens	<100ms	Low latency

Implementation Checklist

- ☐ Package model weights (< 20GB recommended)
 - ☐ Create inference script with error handling
 - ☐ Add input validation and rate limiting
 - ☐ Set up logging and monitoring
 - ☐ Configure auto-scaling parameters
 - ☐ Create health check endpoint
 - ☐ Test cold start performance
 - ☐ Set up alerting for errors/latency
 - ☐ Document API request/response format
 - ☐ Create load test to verify scaling
-

Recommended Setup

For your IMDb QLoRA model:

1. **Primary:** Use Modal for development and production
 - Warm pools prevent cold starts
 - Easy scaling and versioning
 - Built-in monitoring
 2. **Backup:** Deploy to Google Cloud Run for redundancy
 - Multi-region support
 - Simple containerization
 3. **Monitoring:** Use Prometheus + Grafana
 - Track inference latency
 - Monitor GPU/CPU utilization
 - Alert on error rates
-

Getting Started

1. Choose a platform (recommend Modal for fastest setup)
2. Create inference wrapper around your fine-tuned model
3. Deploy and test with sample prompts
4. Add monitoring/alerting
5. Set up auto-scaling based on queue depth